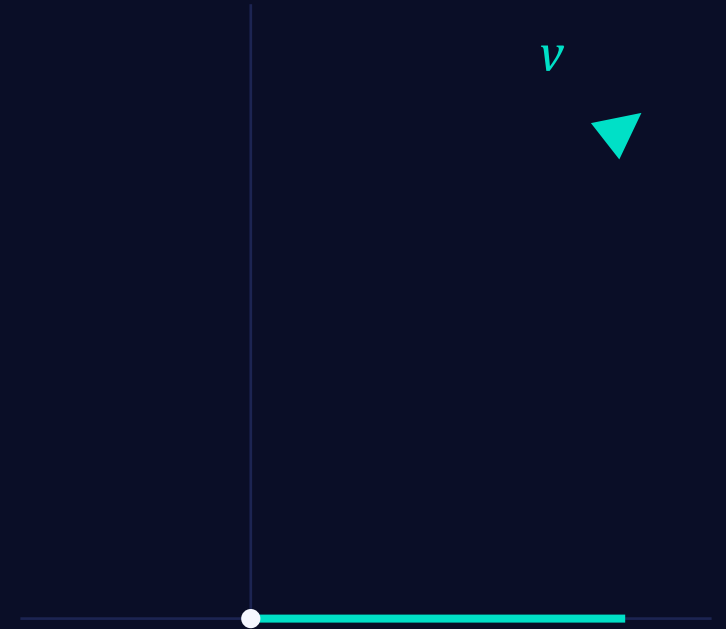


Vector Mathematics

Module 2 — Session 5

From Position to Direction



From OOP to Vector Movement

Last session, we refactored our Space Shooter into OOP.



Our game is now organized.

But how should objects move?

Movement With X & Y

Previously, movement was handled directly through the Rect.

```
self.rect.x += self.speed * dt  
self.rect.y -= self.speed * dt
```

This works well for simple movement. But what if we want:

8-direction movement

Direction

Distance

Smooth movement

Movement toward another
object

We need a better way to represent movement.

What Is a Vector?

A vector can represent:

POSITION

```
position = pygame.Vector2(400, 300)
```

DIRECTION

```
direction = pygame.Vector2(1, 0)
```

A Vector2 contains two numbers: X and Y

READING A DIRECTION

(1, 0)



Right

(-1, 0)



Left

(0, -1)



Up

(0, 1)



Down

pygame.Vector2

Pygame provides Vector2 for 2D game mathematics.

```
position = pygame.Vector2(400, 300)

print(position.x)
print(position.y)
```

Vectors can also be added, subtracted, and multiplied.

```
position += pygame.Vector2(5, 0)
```

VECTOR ADDITION

(400, 300)

+

(5, 0)



(405, 300)

Separating Position and Direction

Instead of directly changing the Rect:

```
self.rect.x += ...
```

We separate it into two ideas:

```
self.position = pygame.Vector2(400, 500)  
  
direction = pygame.Vector2(1, 0)
```

POSITION

"Where am I?"

DIRECTION

"Where am I going?"

+

Direction × Speed × Time

```
self.position += direction * self.speed * dt
```

direction

Which way?

×

speed

How fast?

×

dt

How much time passed?



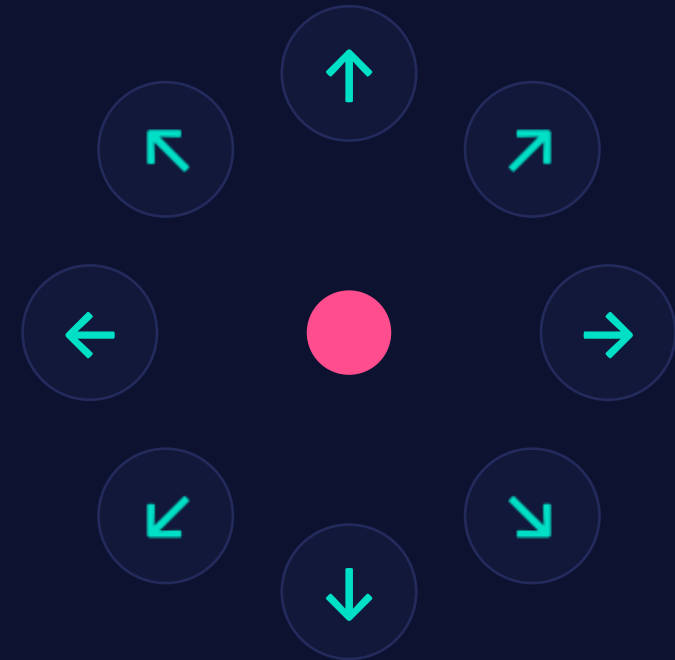
Movement

Moving With Vector2

```
direction = pygame.Vector2(0, 0)

if keys[pygame.K_LEFT]:
    direction.x -= 1
if keys[pygame.K_RIGHT]:
    direction.x += 1
if keys[pygame.K_UP]:
    direction.y -= 1
if keys[pygame.K_DOWN]:
    direction.y += 1
```

ONE VARIABLE, EIGHT DIRECTIONS



direction

The Diagonal Problem

VECTOR LENGTHS

Right	(1, 0)	length 1
Down	(0, 1)	length 1
Diagonal	(1, 1)	length $\sqrt{2} \approx 1.41$

Diagonal movement can become faster than horizontal or vertical movement.

We fix this with:

```
if direction.length() > 0:  
    direction = direction.normalize()
```

normalize()

Keep the direction, but make its length 1.



Vector2 and Rect Have Different Jobs

We don't have to throw away Rect.

Vector2

- Position
- Movement
- Direction

Rect

- Drawing
- Collision
- Size

```
self.position += direction * self.speed * dt  
  
self.rect.center = self.position
```

Vector controls movement. **Rect handles the game object's area.**

Upgrade the Space Shooter

Take the Space Shooter from Session 4.

REQUIRED

- Use pygame.Vector2
- Player has a Vector2 position
- Player uses direction
- Movement uses $\text{speed} \times \text{dt}$
- Normalize diagonal movement
- Rect still handles collision/drawing
- Player uses the new spaceship asset
- Asteroids use image assets
- Space background is used

CHALLENGE

Apply Vector2 movement to:

Player · **Laser** · **Asteroid**

GOAL

Same Space Shooter — better movement architecture.

Today's Big Ideas

Vector2



Position & Direction



Movement



Direction × Speed × dt

```
direction = pygame.Vector2(0, 0)
direction = direction.normalize()
position += direction * speed * dt
```

NEXT SESSION

Kinematics & Friction

How do we make movement feel less robotic?

Velocity



Acceleration



Friction